Assignment 2 for E0 259: Data Analytics

Chinmay Jain (SR No: 23006) * 1

1. Introduction

This report details the optimization of a Mars equant model, a geometric model from classical astronomy used to describe planetary motion, particularly Mars' opposition. The objective was to optimize orbital parameters (r, s, c, e1, e2, z) by minimizing angular errors between the calculated and observed oppositions of Mars. The model was implemented and optimized using Python with *numpy* for numerical computations and *multiprocessing* for parallel processing.

2. Datasets

2.1. Tycho Brahe's Observations:

The dataset used includes the dates and times of 12 Mars oppositions, along with the corresponding zodiac degrees.

2.2. Data Overview

The dataset was provided as an list of oppositions with the columns as :

- **Time Data** (Columns A-E): Year, Month, Day, Hour, Minute of opposition.
- **Position Data** (Columns F-I): Zodiac index, degree, minute, and second of Mars's longitude in the ecliptic coordinate system.
- **Geocentric Data** (Columns J-K): Geocentric latitude .
- **Mean Longitude (Columns L-O)**: These columns refer to Kepler's approximated equant.

3. Libraries and Attributes Used

- `NumPy`: Used for numerical computations like array manipulation and matrix operations.
- `Multiprocessing (optional)`: Used for parallelizing the betweenness centrality calculation in the Girvan-Newman algorithm for improved performance on larger datasets.
- `datetime`: This library is used to work with dates and times.Here I have used this to calculate the difference in days and seconds between the opposition dates and the reference date.

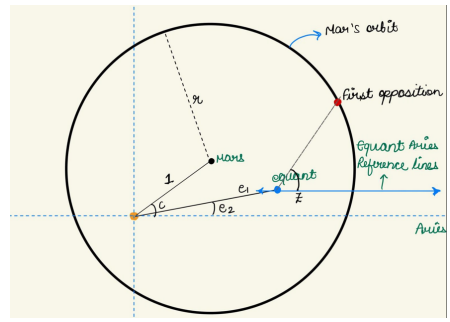

Figure 1. Mars Plot

- `math`:This library provides mathematical functions such as `cos, sin, tan, atan2,degrees, radians` which are used for trigonometric calculations and conversions between radians and degrees.

4. `Implementation:`

4.1. Model Parameters

- **Orbit Center** : Distance from the Sun set to 1 unit, at an angle c from the Sun-Aries reference line.
- **Mars's Orbit Radius (r)**: The equant is placed at polar coordinates (e1, e2), where e1 is the radial distance and e2 is the angle from the Sun-Aries line.
- **Reference Angle (z)**: The angle z is taken as the position at the first opposition.
- **Angular Velocity (s)**: Mars's angular velocity around the equant in degrees per day.

4.2. Function 1:`MarsEquantModel`

This is the core function which calculates the angular error for each opposition using the input parameters to the function: c, r, e1, e2, z, s, times, and oppositions.The function `MarsEquantModel` simulates Mars's orbit using the equant-based model. It calculates the predicted positions of Mars during opposition and the angular errors between the predicted and observed positions for each opposition.

Input:

- c: Angle (in degrees) of Mars's orbit center from the Sun-Aries line.
- r: Radius of Mars's orbit.
- e_1, e_2: Polar coordinates of the equant with respect to the Sun.
- z: The reference angle taken at the earliest opposition.
- s: Angular velocity (degrees per day) of Mars around the equant.
- `times`: Numpy Array of time values (in days) for each opposition.
- `oppositions`: Numpy Array of observed longitudes (in degrees) of Mars for each opposition.

Output:

- `errors`: Array of angular errors (in degrees) for each opposition.
- `maxError`: Maximum angular error from the set of errors.

Explanation and Mathematics involved:

4.2.1. Step 1: Finding the Slope of Mars' Orbit at Time t

Mars moves with an angular velocity s (in degrees per day), starting from an initial angular position z at time $t = 0$. After time t, the angular position of Mars with respect to the equant is:

$$\text{slope} = (z + s \times t) \mod 360$$

where,

- z: Reference angle at $t = 0$, relative to the equant-Sun-Aries line.
- s: Angular velocity (degrees per day).
- t: Time elapsed (days).

This angle is the direction of Mars' position with respect to the equant point in polar coordinates.

4.2.2. Step 2: Finding the Equation of the Line passing from the Equant

In polar coordinates, the equant point is located at (e_1, e_2), where:

- e_1: Distance from the Sun to the equant.
- e_2: Angle of the equant point with respect to the Sun-Aries line.

We convert the equant point to Cartesian coordinates (x_1, y_1) using:

$$x_1 = e_1 \cos(e_2), \quad y_1 = e_1 \sin(e_2)$$

The line passing through the equant point (x_1, y_1) at an angle slope has a slope m given by:

$$m = \tan(\text{slope})$$

The equation of the line in slope-intercept form is:

$$y = mx + c$$

where c (the y-intercept) is given by:

$$c = y_1 - m \cdot x_1$$

4.2.3. Step 3: Finding the Intersection of the Line and Mars' Orbit

Mars' orbit is assumed to be a circle with center at (h, k) and radius r. The center (h, k) is in polar coordinates at distance 1 from the Sun, with an angular position c, which is converted to Cartesian coordinates:

$$h = \cos(c), \quad k = \sin(c)$$

The equation of the circle is:

$$(x - h)^2 + (y - k)^2 = r^2$$

Substituting the line equation $y = mx + c$ into the circle equation gives:

$$(x - h)^2 + (mx + c - k)^2 = r^2$$

Expanding this results in a quadratic equation in x:

$$Ax^2 + Bx + C = 0$$

where:

$$A = 1 + m^2$$
$$B = -2h + 2m(c - k)$$
$$C = h^2 + (c - k)^2 - r^2$$

4.2.4. Step 4: Solving for the Intersection Points

The quadratic equation:

$$Ax^2 + Bx + C = 0$$

has solutions for x (the intersection points) given by:

$$x_1 = \frac{-B + \sqrt{B^2 - 4AC}}{2A}, \quad x_2 = \frac{-B - \sqrt{B^2 - 4AC}}{2A}$$

The corresponding y-coordinates are found using the line equation:

$$y_1 = mx_1 + c, \quad y_2 = mx_2 + c$$

4.2.5. Step 5: Choosing the Correct Intersection Point

We calculate the angular position of each intersection point (x_1, y_1) and (x_2, y_2) relative to the Sun-Aries line. The angular position (or slope) of a point (x, y) is:

$$\theta = \arctan\left(\frac{y}{x}\right)$$

The correct intersection point is the one whose angular position is closest to the observed opposition angle θ_{obs}.

4.2.6. Step 6: Calculating the Angular Error

The error for each time t is the absolute difference between the predicted angular position of Mars and the observed opposition angle:

$$\text{error} = |\theta_{\text{pred}} - \theta_{\text{obs}}|$$

where:

- θ_{pred}: Predicted angular position (from the intersection).
- θ_{obs}: Observed opposition angle.

This process is repeated for each time t, and the maximum error across all times is recorded.

4.3. Function 2: Search for Best Parameters (Fixed r and s)

The second problem involves finding the best parameters (c, e_1, e_2, z) that minimize the maximum angular error for a fixed orbit radius (r) and angular velocity (s). This is solved using a discretized exhaustive search:

Parameter Search Range:

- c: Varies between $145°$ and $160°$ (discretized over 100 steps).
- e_1: Varies between 1 and 2 (discretized over 100 steps).
- e_2: Varies between $145°$ and $160°$.
- z: Varies between $45°$ and $60°$.

Each combination is evaluated, and the parameters that minimize the maximum angular error are selected.

Function 3: Search for Best s (Fixed r)

In this problem, the search is extended to find the best angular velocity (s) for a fixed orbit radius (r). The parameter search involves:

Parameter Search Range:

- s: Varies around $360°/687$ days $\pm$ small variations, discretized into 200 steps.

For each value of s, the best set of inner parameters (c, e_1, e_2, z) is recalculated to minimize the maximum angular error.

Function 4: Search for Best r (Fixed s)

Here, we attempt to find the best orbit radius (r) by searching around an initial guess. The approach involves:

Parameter Search Range:

- r: Varies between 8 and 10 units (discretized into 200 steps).

For each r, the best values of the angular velocity (s) and inner parameters (c, e_1, e_2, z) are determined to minimize the angular error.

Function 5: Full Iterative Search Over r and s

Finally, the function `bestMarsOrbitParams` iteratively searches over both r and s, starting from an initial guess. The process alternates between optimizing r and s to converge on the best orbit parameters.

The output of this function includes:

- The best values of r, s, c, e_1, e_2, and z.
- The angular errors for each opposition.
- The maximum angular error.

4.4. Multiprocessing Optimization

The parameter search in the Mars Orbit Model is optimized using **multiprocessing** to speed up the computation. By dividing the exhaustive search across multiple CPU cores, the program evaluates different parameter combinations concurrently. Using Python's `multiprocessing.Pool`, tasks are distributed to worker processes, significantly reducing the total computation time. This allows for faster optimization of orbit parameters like r, s, c, e_1, e_2, and z, making the search process more efficient without compromising accuracy.

Results

The optimized parameters obtained after running the `bestMarsOrbitParams` function on the provided dataset are as follows:

- **Best Radius (r):** 9.4796 units

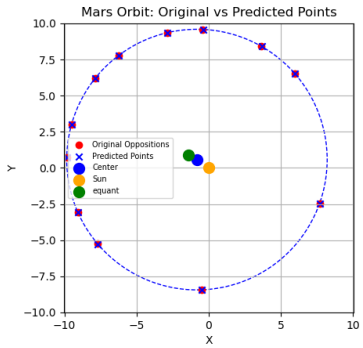

Figure 2. Actual Vs Predicted Points

- **Best Angular Velocity (s):** 0.5241°/day
- **Best c:** 143.7288°
- **Best Equant Coordinates:**
  - e_1: 1.7551 units
  - e_2: 148.8136°
- **Best z:** 55.932°

The maximum angular error obtained using these parameters is approximately 0.0790°, which is quite small, indicating a good fit for the observed oppositions.